

Country Recommendation System Using Graph Embedding

Node Classification via PCA + Graph Features + Random Forest

How to Run · Google Colab · Python 3

What is This Project?

This project builds a **country recommendation and classification system** using Graph Machine Learning. It takes LinkedIn user data for 227 countries and constructs a **similarity graph** where each country is a node and edges connect countries with similar adoption profiles. It then extracts 20 graph-based features per node, applies PCA for dimensionality reduction, and trains a Random Forest to classify each country into one of four behaviour classes: **Emerging**, **Growing**, **Established**, or **Saturated**.

Quick Facts

Dataset: 227 countries — LinkedIn users 2023 & 2024

Graph: Cosine similarity graph, threshold 0.85, ~5,145 edges

Features: 20-dimensional vector per node (graph metrics + k-hop + engineered)

Classes: 4 behaviour classes assigned by penetration quartile

Best result: 98.25% test accuracy | 99.41% cross-validation accuracy

Runs on: Google Colab (free tier) — no local setup needed

Files You Need

You only need **two files** to run this project. Both are provided:

File	Description
<code>linkedin_graph_ml_bda.py</code>	The complete Python pipeline — graph construction, feature engineering, PCA, classifier, and all visualisations
<code>linkedin-users-by-country-2024.csv</code>	The dataset (227 rows, 3 columns: country, LinkedIn users 2024, users 2023 in thousands)

Step-by-Step Instructions

1

Open Google Colab

Go to **colab.research.google.com** and click *New Notebook*.

You don't need a GPU — the free CPU runtime is sufficient for this project.

✓ Use a Google account to save your notebook automatically to Drive

2

Upload Your Files

In the left sidebar, click the **folder icon** to open the Files panel.

Click the **upload icon** (paper with arrow) and upload both files:

- `linkedin_graph_ml_bda.py`
- `linkedin-users-by-country-2024.csv`

Wait until both files appear in the file browser before continuing.

✓ The CSV filename must match exactly — check for spaces or underscores

3

Install Dependencies

In the first code cell, paste and run this install command:

```
!pip install networkx scikit-learn pandas numpy matplotlib seaborn --quiet
```

✓ This takes about 30 seconds. You only need to do it once per session

4

Run the Python File

In the next code cell, paste and run:

```
exec(open('linkedin_graph_ml_bda.py').read())
```

✓ Alternatively, copy-paste the entire contents of `linkedin_graph_ml.py` directly into cells

5

Wait for Output

The script runs in sections. You will see printed progress for each section:

- Graph construction (~5 sec)
- Graph metrics — betweenness centrality is slowest (~10–30 sec)
- Feature matrix building (~15 sec)
- PCA analysis + classifier training (~5 sec)
- 6 visualisation figures (appear inline in Colab)

✓ Total runtime: approximately 1–2 minutes on Colab free tier

6

Download Your Results

All 6 figures are saved as PNG files in Colab's working directory.

To download them: right-click each file in the Files panel → Download.

Files saved:

- fig1_pca_variance.png • fig2_pca_embeddings.png
- fig3_confusion_matrices.png • fig4_feature_importance.png
- fig5_cv_comparison.png • fig6_graph_overview.png

✓ Use File > Download > Download .ipynb to save your full notebook

Troubleshooting

Problem	Fix
<code>FileNotFoundError: CSV not found</code>	The CSV filename in the Files panel must exactly match <code>linkedin-users-by-country-2024.csv</code> — check for underscores vs hyphens
<code>ModuleNotFoundError: No module named 'networkx'</code>	Re-run Step 3 (the pip install cell). Colab resets installed packages when the session restarts
Runtime disconnects mid-run	Colab disconnects after ~90 min of inactivity. Re-run from Step 3. The script is fully re-runnable
Figures don't appear inline	Add <code>%matplotlib inline</code> at the top of the cell before running the script

Understanding the Output

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Figure' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='He lvetica-Bold', fontSize=10, greek=0, italic=0, link=[], rise=0, text='Figure', t extColor=Color(1,1,1, 1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'What It Shows' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=10, greek=0, italic=0, link=[], rise=0, text='What It Shows', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'What to Look For' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=10, greek=0, italic=0, link=[], rise=0, text='What to Look For', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph
Fig 1 — PCA Variance	Scree plot + cumulative variance curve	<i>How many PCs cover 90/80/60% variance</i>
Fig 2 — PCA Embeddings	2D scatter of first two PCs per threshold	<i>Class separation in low-dimensional space</i>
Fig 3 — Confusion Matrices	Prediction vs. actual class for all 4 configs	<i>Which classes are confused and by how much</i>
Fig 4 — Feature Importance	Gini importance of all 20 features	<i>Which feature group (graph/node/k-hop) drives accuracy</i>
Fig 5 — CV Comparison	Cross-validation accuracy with error bars	<i>Model stability across folds (lower std = more reliable)</i>
Fig 6 — Graph Overview	Spring-layout of top-30 degree-central nodes	<i>Community structure and class clustering in graph</i>

Note: The DATA_PATH variable at the top of Section 2 in the .py file must match your CSV filename exactly. If you rename the CSV, update that one line. The rest of the code is self-contained and requires no other changes to run end-to-end.